? Plano de implementação ? Matchmaking compartilhado + Inter-servidores +

Origem: feature de fila compartilhada entre servidores (N x J na CAPS ARENA neutra).
Restrições: modular, arquivos < 600 linhas, baixa complexidade ciclomática, sem race conditions, paralelismo para dezenas/centenas de filas simultâneas, verificação independente ao final.

Estado atual (base pronta)

- Inter MD1/MD3 na arena: implementado, 80/80 testes (`domain/inter/`, `application/use-cases/inter/`, `services/inviteService.js`).
- Painel local `admin/` (Express+EJS, sem login, sem temas) ? insuficiente para gestão multi-guild.
- Lock em memória (`withQueueOperationLock`) ? 1 instância; `config.json` global por ambiente.

FASE 0 ? Preparação (1 sessão)

0.1. Congelar escopo do MVP (seção MVP abaixo) e abrir branch `feat/shared-matchmaking`.
0.2. Backup código + banco (`backups/`, `migrateToMongo.js` de volta se preciso).
0.3. Definir matriz de guilds: N, J, ARENA (IDs) + quem tem Gerenciar Servidor em cada uma.
0.4. Criar épicos/issues por fase para delegação paralela.

FASE 1 ? Fundação de dados e concorrência

1.1. `guildSettings` persistido: `{guildId: {sharedMatchmaking, sharedGroupId=null, interAtivo, inviteChannelId, announceChannelId, capitaoRoleId}}` + load/save em `services/dataService.js` (novo módulo `services/guildSettingsService.js` se passar de ~150 linhas).
1.2. Carimbar origem: `lobby.players[].originGuildId`, `lobby.{scope, destinationGuildId}`, `match.playerOrigins` (modo interno compartilhado).
1.3. Lock distribuído por chave (lease no Mongo, TTL 5?10s, dono+expiração) em `services/distributedLock.js`, mantendo a API `withQueueOperationLock(key, fn)`. Chaves: `queue:<mode>:<format>` (shared), `inter:global:busca`, `inter:match:<id>`, `<guildId>:victory`.
1.4. `systemMeta.lastQueueMessageId` ? mapa por guild (`dashboardMeta {guildId: messageId}`).
1.5. Testes: `guild-settings`, `distributed-lock` (lease, expiração, dois workers simulados), `dashboard-meta`.

FASE 2 ? Núcleo do matchmaking compartilhado

2.1. Pool shared por `shared:<mode>:<format>` para guilds com `sharedMatchmaking=true`; guilds `false` usam caminho atual intacto.
2.2. Anti-duplicata global: checar `discordId` em lobbies + waitingLists + matches ativos + buscas inter antes de entrar.
2.3. Eleição de destino: maioria de origem ? desempate (quem completou / round-robin / menor carga) ? `preferredHostGuildId` opcional.
2.4. Timeouts/cancelamentos cross-guild: `!sair` remove de qualquer pool; dashboards atualizados em todas as guilds participantes; `!shared leave` (staff) retira pendentes da guild.
2.5. Testes: `shared-queue` (2 guilds enchem 1 lobby), `destination`, `concurrency` (entradas paralelas A+B), `leave-cross-guild`.

FASE 3 ? Invites e notificações (base pronta, estender)

3.1. `services/inviteService.js`: reutilizar; adicionar renovação de invite expirado + `inviteDelivery {sent, dmFailed}` por match.
3.2. Fallbacks: DM fechada ? menção pública; sem permissão ? rollback sem canal órfão (padrão já implementado no inter).

3.3. Testes: `invite-renew`, `invite-rollback` (mocks).

FASE 4 ? Inter N x J na ARENA (base pronta)

4.1. Manter buscas FIFO, MD1/MD3, sorteio J3, ranking separado, W.O. (já implementado).
4.2. Grupos explícitos fase 2: `sharedGroupId` filtra pares de buscas (só mesmo grupo se forma match).
4.3. `setup-servidor.js`: passar a gravar `inter.*`/`guildSettings` por guild em vez de sobrescrever o global.
4.4. Testes: já existem 20; adicionar `shared-group` quando 4.2 entrar.

FASE 5 ? Dashboard web administrativo (novo)

Situação: `admin/` local sem login não serve para gestão multi-guild. Construir `admin/` v2:

5.1. **Auth local (sem segredo em chat/commit):** script `admin/create-admin.js` executado **pelo dono no servidor**, que grava hash bcrypt em coleção `adminUsers` (usuário inicial criado na primeira execução via terminal). Sessão por cookie httpOnly. Níveis: `admin` (tudo) e `viewer` (só leitura).
5.2. **Temas claro/escuro:** toggle com CSS variables + persistência em `localStorage`; sem framework novo (EJS + CSS próprio, como hoje).
5.3. **Telões (leitura):** filas por guild/modo, matches ativos, buscas inter, séries recentes, ranking inter + interno, jogadores (busca por nick/discordId), logs de temporada, saúde (uptime, DB, locks, erros recentes).
5.4. **Ações (admin):** cancelar busca, aplicar W.O., fechar série, remover jogador da fila, reenviar invite, editar `guildSettings` (on/off shared, canais, pontos), exportar backup.
5.5. **Limites de módulo:** um arquivo por tela em `admin/routes/`, um por serviço em `admin/services/`; nenhum > 600 linhas; sem regra de negócio (só chama use-cases).
5.6. Testes: `admin-auth` (login/logout, níveis), `admin-actions` (cada ação via use-case mockado).

FASE 6 ? Endurecimento e escala

6.1. Particionar chaves de lock por `mode:format` (paralelismo real entre filas distintas).
6.2. Fila de jobs (em-processo primeiro: `workers/` com retry/backoff) para Riot API, anúncios e limpeza.
6.3. Rate limit + circuit breaker na Riot API.
6.4. Observabilidade: log estruturado por comando/match (`correlationId`), métricas de fila (tempo médio até formar).
6.5. Teste de carga simulada: dezenas de filas formando em paralelo sem lost update.

FASE 7 ? Verificação independente

Agente/revisor separado (outra sessão ou pessoa) executa o checklist e só aprova com tudo verde:

- [ ] Cada requisito do MVP tem teste automatizado correspondente (mapear requisito?teste).
- [ ] `npm test` 100% verde; nenhum arquivo de código > 600 linhas (`wc -l` auditado).
- [ ] Sem `grep` de `createInvite` fora de `inviteService.js`; sem webhook improvisado.
- [ ] Simulação de corrida (2 entradas simultâneas) sem duplicata nem overfill.
- [ ] Fluxo manual na TESTRE: busca ? match ? invite ? resultado ? ranking ? limpeza.
- [ ] Dashboard: login, temas, telas e ações conferidos; nenhum segredo no repo (`git grep` por token/senha).
- [ ] Relatório de achados com severidade; bugs críticos bloqueiam o merge.

?? Recorte do MVP (entrega mínima com valor)

Entra no MVP:
- Inter MD1/MD3 na arena com 5 fixos (pronto) + elenco pela call (pronto).
- Opt-in por guild (`sharedMatchmaking` em `guildSettings`; default off, interno intacto).
- Pool shared por `mode:format` + eleição de destino por maioria + invite só no match pronto.
- Ranking inter separado + W.O. + comandos texto/slash + `!ajuda`/`!staff`.
- Dashboard v1: **login local + leitura** (filas, matches, buscas, ranking, logs) + temas claro/escuro + ação única: cancelar busca.
- Lock atual (1 instância) com aviso documentado; testes de concorrência em-processo.

Fica para depois (pós-MVP):
- Grupos explícitos (`sharedGroupId`), lock distribuído multi-instância, fila de jobs/BullMQ, rate limiter Riot, ações avançadas do dashboard (WO, edição de settings, export), métricas e carga.

Critério de pronto do MVP: checklist da Fase 7 aplicado só ao escopo acima + teste funcional na TESTRE com 2 times de 5.

? Delegação paralela sugerida (subagentes)

Frente	Escopo	Independente de
A. Núcleo shared (Fase 2)	pool, destino, anti-duplicata	? (base)
B. Lock distribuído (1.3)	`distributedLock.js` + testes	A (contrato da API mantido)
C. Dashboard UI (5.2?5.3)	telas leitura + temas	A/B
D. Dashboard ações+auth (5.1, 5.4)	login, cancelar busca	C (rotas)
E. Testes e2e/stress (6.5, Fase 7 parcial)	cenários e scripts	A?D
F. Docs/help (ajuda, STAFF, MEMORY)	textos	A

A aguarda B apenas na integração final (troca do lock). C, D, E, F rodam em paralelo após os contratos (assinaturas de use-cases) congelados na Fase 0.

? Credenciais (procedimento seguro)

- Nunca gerar nem compartilhar senhas/tokens em chat, issue ou commit.
- O dono executa `node admin/create-admin.js <usuario>` **no servidor do bot**; o script pede a senha no terminal e grava só o hash.
- Tokens de bot continuam em `.env` (ignorado pelo git). Rotação: gerar novo no Portal ? trocar no `.env` ? `pm2 restart --update-env`.